

ISA 345 Project Report
Fitness Studio Database

Lauren Rudek, Morgan Tanking, Ellie Ploszay, Khushi Shah, & Jazelle Fraise

April 30, 2026

Project Overview

Problem and Domain

The purpose of this project is to design and implement a relational database system to support the operations of a multi-location fitness studio modeled after the Tracy Anderson training concept. The business environment consists of customers enrolling in membership plans, booking fitness classes, attending scheduled sessions, and making payments through various methods. As the business scales across multiple studios and instructors, the need for an efficient, centralized data management system becomes critical.

The primary problem addressed by this database system is the lack of integration between customer data, booking systems, instructor scheduling, and financial transactions. Without a structured database, the organization would face inefficiencies in tracking attendance, monitoring revenue streams, managing memberships, and evaluating instructor performance. These challenges could lead to poor decision-making, reduced customer satisfaction, and lost revenue opportunities.

To address these issues, the proposed database system integrates all core business functions into a unified relational structure. The system enables seamless management of operational transactions such as booking fitness sessions, processing payments, and managing membership subscriptions. For example, when a customer books a class, the system records the booking, associates it with a specific session, and links it to the customer's membership details. Similarly, payment transactions are recorded and tied directly to customer profiles, enabling accurate financial tracking. Additionally, booking cancellations are handled through controlled updates, ensuring data consistency and enabling partial refund processing through procedural logic.

Core business processes:

Class and Session Management: Defining different class types and scheduling specific sessions in various studios with assigned instructors.

Instructor Assignment: Managing instructors and linking them to specific studios and sessions.

Customer Membership Management: Handling different membership plans, customer profiles, and active subscriptions.

Booking and Reservation System: Allowing customers to book sessions while enforcing studio capacity limits and real-time availability.

Payment Processing: Recording and tracking customer payments linked to memberships or individual bookings.

Studio Operations: Overseeing multiple physical studio locations with their capacity and contact details.

Operational Transactions

We also have operational transactions. The first operational transaction would be a customer booking a class session. When a customer decides to book a class the database first checks the session capacity, then inserts a record into the BOOKING table, and finally updates the session enrollment.

Customer Booking a Class Session:

The system checks session capacity → inserts a booking record → updates session enrollment

The second operational transaction would be a customer purchasing a membership. When a customer decides to purchase a membership a MembershipID is assigned to them, then a record is added to the PAYMENT table, and finally the membership pricing and plan details will be stored.

Customer Purchasing a Membership:

Assigns MembershipID → records payment → updates membership details

The last operational transaction would be processing a payment whether it is for a single class booking or purchasing a membership. When the payment is made it is inserted into the PAYMENT table, then that payment is linked to a specific customer, and finally the method and billing date is recorded.

Processing a Payment:

Inserts payment record → links to customer → records method and billing date

Database Design and Implementation

Entities and Attributes

Our database is compiled of eight entities, each with multiple attributes.

Class - types of fitness classes offered

- ClassID - PK. Unique identifier for the class type
- ClassType - Category name of the class (Yoga, Spin, Pilates, Barre)
- Description - Description of class format, goals, and details
- DifficultyLvl - Difficulty rating of the class (Beginner, Intermediate, Advanced)
- ClassDurationMins - Length of class in minutes
- Equipment - Equipment needed to be used by customers (Mat, Reformer, Weights, Ball)

Session - scheduled occurrences of the classes

- SessionID - PK. Unique identifier of scheduled session
- ClassID - FK. The type of class being referenced in this session
- InstructorID - FK. The instructor teaching the class.
- SessionDate - Date the session is being held
- SessionTime - Time the session is being held
- StudioID - FK. Location where the session is being held

Studio - physical locations

- StudioID - PK. Unique identifier for the studio
- StudioCity - City where the studio is located in
- Address - Complete street address of the studio
- MaxCapacity - Maximum number of customers that can be in the studio
- StudioPhoneNum - Phone number for the studio
- StudioCountry - Country the studio is located in

Instructor - fitness instructors

- InstructorID - PK. Unique identifier for the instructor
- InstructorName - Full name of the instructor
- StudioID - FK. Instructors home studio

Payment - financial transactions

- PaymentID - PK. Unique identifier for payments
- PaymentMethod - What was used for payment (Apple Pay, Credit/Debit, Cash)
- CustomerID - FK. The customer who made the payment
- BillingDate - Date that the payment was processed
- PaymentAmt - Amount charged for the payment

Customer - customers of the workout studio

- CustomerID - PK. Unique identifier for the customer
- CustomerName - Name of the customer
- CustomerPhoneNo - Customers phone number
- MembershipID - FK. Membership plan the customer is on

Membership - membership plans

- MembershipID - PK. Unique identifier of the membership plan
- PlanName - Name of the membership plan (Basic, Premium, Unlimited)
- Description - Explanation of what is included in the plan
- MembershipPrice - Cost of the membership

Booking - class reservations

- BookingID - PK. Unique identifier of the booking
- CustomerID - FK. Customer who made the booking reservation
- SessionID - FK. The session that is being booked
- BookingDate - Date that the booking was made
- BookingStatus - Status of the booking (Confirmed, Waitlisted, Cancelled)

Relationships and Business Rules

STUDIO - INSTRUCTOR (1:M)

- Each studio employs zero to many instructors

- Each instructor is assigned to exactly one studio
- An instructor must belong to a studio (no unassigned instructors allowed)

STUDIO - SESSION (1:M)

- Each studio hosts zero to many sessions
- Each session takes place at exactly one studio
- A session cannot exist without a studio

CLASS - SESSION (1:M)

- Each class type can be offered in zero to many sessions
- Each session is associated with exactly one class
- A session must reference a valid class type

INSTRUCTOR - SESSION (1:M)

- Each instructor may lead zero to many sessions
- Each session is led by exactly one instructor
- Instructors may teach at studios other than their home studio

MEMBERSHIP - CUSTOMER (1:M)

- Each membership plan may be held by zero to many customers
- Each customer has exactly one membership plan at a time
- Customers must have a membership (no non-members allowed)

CUSTOMER - BOOKING (1:M)

- Each customer may create zero to many bookings
- Each booking belongs to exactly one customer
- A booking must be associated with a valid customer

SESSION - BOOKING (1:M)

- Each session may have zero to many bookings
- Each booking references exactly one session

A booking must reference a valid scheduled session

CUSTOMER - PAYMENT (1:M)

- Each customer has one to many payment records (at least one required)
- Each payment belongs to exactly one customer
- Every active customer must have at least one payment on record

Normalization and Functional Dependencies

For normalization, each entity was evaluated against First Normal Form 1NF, Second Normal Form 2NF, Third Normal Form 3NF, and BCNF. For Functional Dependencies, the ones involving primary keys were identified and corrected before any implementation happened.

First Normal Form 1NF - All attributes include no repeating groups.

Second Normal Form 2NF - All attributes that are not keys are fully dependent on the primary key.

Third Normal Form 3NF - there are no transitive dependencies.

BCNF - Each determinant is a candidate key. We made sure keys were reviewed to make sure that no non-key attributes determine another non-key attribute.

Key functional dependencies

ClassID → ClassType, Description, DifficultyLvl, Duration, Equipment

SessionID → ClassID, InstructorID, SessionDate, SessionTime, StudioID

CustomerID → CustomerName, CustomerPhoneNum, MembershipID

MembershipID → PlanName, Description, MembershipPrice

BookingID → CustomerID, SessionID, BookingDate, BookingStatus

PaymentID → CustomerID, PaymentMethod, BillingDate, PaymentAmt

Physical Model

The physical model was created using Oracle SQL Developer.

- Each entity was created as a relational or logical table and used specific data types like VARCHAR, NUMBER, DATE and using correct field sizes when needed
- Primary and foreign keys were defined for all tables
- Constraints like NOT NULL were added

Implementation Notes

We used ID based primary keys, foreign keys, validation rules, and indexing to keep the database concise and accessible.

Problems encountered

During implementation, a technical issue was encountered related to the foreign key constraint between the Booking and Sessions tables due to naming inconsistencies. This issue was resolved by redefining the constraint to correctly reference the “SESSIONS” table, ensuring successful enforcement of referential integrity. The database was then populated with realistic sample data, with at least ten records per table, to support meaningful query execution and analysis.

Operational SQL

Five SQL queries created for our project.

Query 1 - Contains a subquery

The first query identifies high-value and highly engaged customers by combining a subquery with multiple joins. This query enables management to identify customers who spend above average and

actively participate in sessions, making them ideal targets for loyalty programs and premium service offerings.

```
SELECT customername, planname, paymentamt
FROM customer JOIN membership ON customer.customermembershipid = membership.membershipid
JOIN payment ON customer.customerid = payment.paymentcustomerid
WHERE paymentamt > (
    SELECT AVG(paymentamt)
    FROM payment
)
AND customer.customerid IN (
    SELECT bookingcustomerid
    FROM booking
    WHERE bookingstatus = 'Confirmed'
);
```

Query 2 - Contains an innerjoin

The second query employs multiple inner joins to provide a comprehensive view of operational activities. It combines data from bookings, customers, sessions, classes, instructors, and studios to present a detailed record of confirmed bookings. This query enables managers to monitor which customers attended specific classes, who instructed those sessions, and where they took place. Such visibility is essential for resource allocation, scheduling optimization, and performance tracking across different locations.

```
SELECT customername, classtype, sessiondate, instructorname, studiocity
FROM booking JOIN customer ON booking.bookingcustomerid = customer.customerid
JOIN "SESSIONS" ON booking.bookingsessionid = sessions.sessionid
JOIN class ON sessions.sessionclassid = class.classid
JOIN instructor ON sessions.sessioninstructorid = instructor.instructorid
JOIN studio ON sessions.sessionstudioid = studio.studioid
WHERE bookingstatus = 'Confirmed';
```

Query 3 - Contains an outerjoin

The third query incorporates a left outer join to identify customers who have not made any bookings. By including all customers regardless of booking activity, the query highlights inactive members who may be at risk of churn. This information is critical for marketing teams seeking to re-engage customers through promotions or personalized outreach efforts.

```
SELECT customername, planname, booking.bookingid
FROM customer JOIN membership ON customer.customermembershipid = membership.membershipid
LEFT OUTER JOIN booking ON customer.customerid = booking.bookingcustomerid
ORDER BY customername;
```

Query 4 - Contains a GROUP BY clause without the HAVING clause

The fourth query uses a GROUP BY clause to aggregate booking data by studio city, providing insights into the geographic distribution of demand. By analyzing the number of bookings per city, management can identify high-performing locations and make informed decisions regarding expansion, marketing investments, or resource allocation.

```
SELECT studiocity, COUNT(booking.bookingid) AS totalbookings
FROM studio JOIN "SESSIONS" ON studio.studioid = sessions.sessionstudioid
JOIN booking ON sessions.sessionid = booking.bookingsessionId
GROUP BY studiocity;
```

Query 5 - Contains a GROUP BY clause together with the HAVING clause

The fifth query extends the use of aggregation by combining GROUP BY with a HAVING clause to identify top-performing instructors. By filtering instructors with more than one confirmed booking, the query highlights individuals who contribute significantly to customer engagement. This information can be used to design incentive programs, allocate high-demand classes, and improve overall service quality.

```
SELECT instructorname, COUNT(booking.bookingid) AS totalbookings
FROM instructor JOIN "SESSIONS" ON instructor.instructorid = sessions.sessioninstructorid
JOIN booking ON sessions.sessionid = booking.bookingsessionId
WHERE bookingstatus = 'Confirmed'
GROUP BY instructorname
HAVING COUNT(booking.bookingid) > 1;
```

Index & View

In addition to these queries, several database optimization and automation features were implemented. Indexes were created on key attributes such as booking customer ID, session ID, and payment customer ID to improve query performance, particularly for join and lookup operations. A view named “customeractivityview” was defined to simplify access to combined customer, membership, booking, and payment data, enabling users to retrieve comprehensive information without writing complex joins. Another view, “popular_memberships,” provides insights into the most subscribed membership plans, supporting strategic pricing and marketing decisions.

```
CREATE INDEX indexbookingcustomer ON booking(bookingcustomerid);
CREATE INDEX indexbookingsession ON booking(bookingsessionid);
CREATE INDEX indexpaymentcustomer ON payment(paymentcustomerid);
```

```
CREATE VIEW customeractivityview AS
SELECT customername, planname, bookingstatus, paymentamt
```

```
FROM customer JOIN membership ON customer.customermembershipid =  
membership.membershipid  
LEFT JOIN booking ON customer.customerid = booking.bookingcustomerid  
JOIN payment ON customer.customerid = payment.paymentcustomerid;
```

```
CREATE VIEW popular_memberships AS  
SELECT Membership.PlanName, COUNT(Customer.CustomerID) AS Total_Customers  
FROM Membership  
INNER JOIN Customer ON Membership.MembershipID = Customer.MembershipID  
GROUP BY Membership.PlanName  
HAVING COUNT(Customer.CustomerID) > 1;
```

Procedure & Trigger

A stored procedure was developed to handle booking cancellations and associated refunds. This procedure updates the booking status and adjusts the customer's payment amount, demonstrating the ability to execute multiple related operations within a single transactional unit. Additionally, a trigger was implemented to enforce business rules during session creation by ensuring that instructors are assigned only to sessions within their designated studios. This trigger enhances data integrity by preventing invalid assignments at the database level.

```
CREATE OR REPLACE PROCEDURE cancelbookingandrefund (  
    refundbookingid VARCHAR2,  
    refundcustomerid VARCHAR2  
) AS  
BEGIN  
  
    UPDATE booking  
    SET bookingstatus = 'Cancelled'  
    WHERE bookingid = refundbookingid;  
    UPDATE payment  
    SET paymentamt = paymentamt - 10  
    WHERE paymentcustomerid = refundcustomerid;  
END;  
/
```

```
CREATE OR REPLACE TRIGGER validateinstructorstudio  
BEFORE INSERT ON "SESSIONS"  
FOR EACH ROW  
DECLARE  
    vinstructorstudio VARCHAR2(10);  
BEGIN
```

```

SELECT instructorstudioid
INTO vinstructorstudio
FROM instructor
WHERE instructorid = :NEW.sessioninstructorid;

IF vinstructorstudio <> :NEW.sessionstudioid THEN
    RAISE_APPLICATION_ERROR(-20004, 'Instructor does not belong to this studio');
END IF;
END;
/

```

Conclusions and Lessons Learned

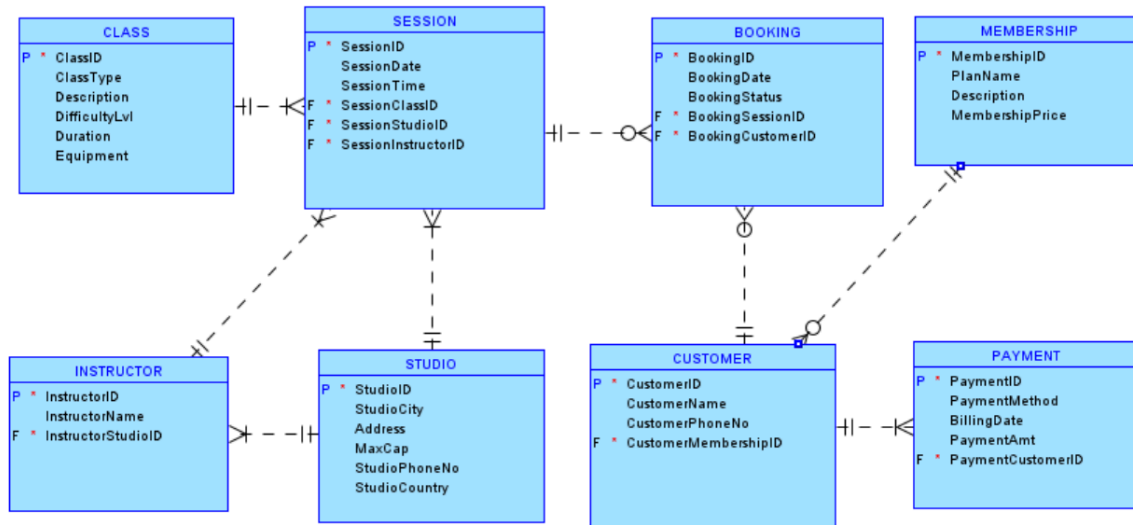
The implemented database system successfully addresses the operational challenges faced by a multi-location fitness studio by providing a structured and integrated data management solution. By consolidating customer information, bookings, payments, and session data into a single relational system, the database enables improved decision-making, operational efficiency, and data accuracy. Managers can now access real-time insights into customer behavior, revenue trends, instructor performance, and location-based demand, allowing for more informed strategic planning.

One of the key benefits of the system is its ability to support advanced analytics through well-designed SQL queries. These queries transform raw data into meaningful insights that can drive business growth, enhance customer engagement, and optimize resource allocation. Additionally, the use of indexes, views, procedures, and triggers demonstrates the system's capability to handle performance optimization, data abstraction, and automated enforcement of business rules.

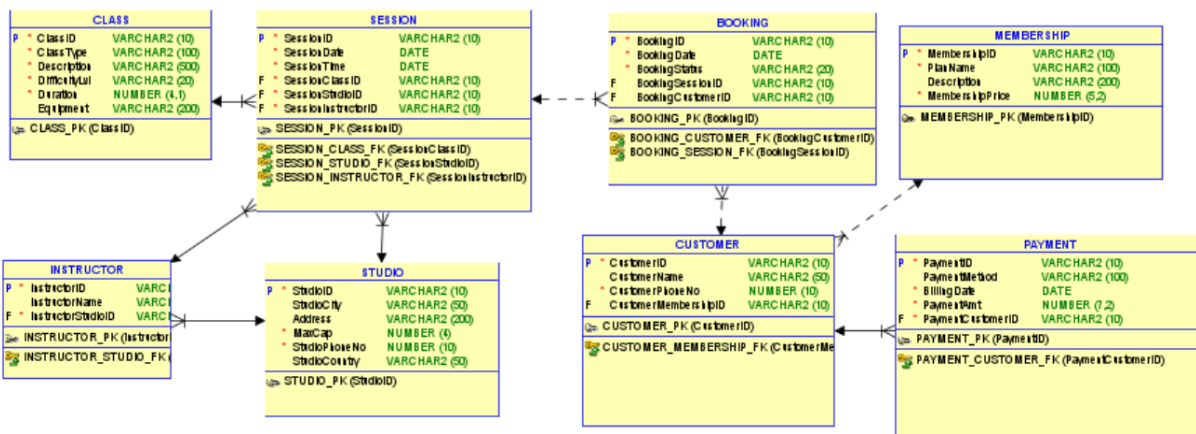
Throughout the project, several challenges were encountered, particularly in ensuring correct implementation of foreign key constraints and maintaining consistency between table definitions. These issues were resolved through careful debugging and validation of SQL statements. The experience highlighted the importance of attention to detail in database design and the need for thorough testing during implementation.

Overall, the project achieved its objectives by delivering a robust and scalable database solution that aligns with the information needs of the business. The system not only improves operational efficiency but also provides a strong foundation for future enhancements, such as integration with web-based applications or advanced reporting tools. This project reinforces the critical role of database systems in modern business environments and demonstrates the practical application of database design principles in solving real-world problems.

Appendix 1: Logical Model



Appendix 2: Relational Database



Appendix 3: SQL Table Queries

```
CREATE TABLE booking (
    bookingid      VARCHAR2(10) NOT NULL,
    bookingdate    DATE NOT NULL,
    bookingstatus  VARCHAR2(20) NOT NULL,
    bookingsessionid VARCHAR2(10),
    bookingcustomerid VARCHAR2(10)
);
```

```
ALTER TABLE booking ADD CONSTRAINT booking_pk PRIMARY KEY ( bookingid );
```

```
CREATE TABLE class (
```

```
classid    VARCHAR2(10) NOT NULL,  
classtype  VARCHAR2(100) NOT NULL,  
description VARCHAR2(500) NOT NULL,  
difficultylvi VARCHAR2(20) NOT NULL,  
duration   NUMBER(4, 1) NOT NULL,  
equipment  VARCHAR2(200)  
);
```

```
ALTER TABLE class ADD CONSTRAINT class_pk PRIMARY KEY ( classid );
```

```
CREATE TABLE customer (  
    customerid    VARCHAR2(10) NOT NULL,  
    customername  VARCHAR2(50),  
    customerphoneno NUMBER(10) NOT NULL,  
    customermembershipid VARCHAR2(10)  
);
```

```
ALTER TABLE customer ADD CONSTRAINT customer_pk PRIMARY KEY ( customerid );
```

```
CREATE TABLE instructor (  
    instructorid  VARCHAR2(10) NOT NULL,  
    instructorname VARCHAR2(50),  
    instructorstudioid VARCHAR2(10) NOT NULL  
);
```

```
ALTER TABLE instructor ADD CONSTRAINT instructor_pk PRIMARY KEY ( instructorid );
```

```
CREATE TABLE membership (  
    membershipid  VARCHAR2(10) NOT NULL,  
    planname      VARCHAR2(100) NOT NULL,  
    description    VARCHAR2(200),  
    membershipprice NUMBER(5, 2) NOT NULL  
);
```

```
ALTER TABLE membership ADD CONSTRAINT membership_pk PRIMARY KEY (  
membershipid );
```

```
CREATE TABLE payment (  
    paymentid    VARCHAR2(10) NOT NULL,  
    paymentmethod VARCHAR2(100),  
    billingdate   DATE NOT NULL,  
    paymentamt    NUMBER(7, 2) NOT NULL,  
    paymentcustomerid VARCHAR2(10) NOT NULL  
);
```

```
ALTER TABLE payment ADD CONSTRAINT payment_pk PRIMARY KEY ( paymentid );
```

```
CREATE TABLE "SESSIONS" (  
    sessionid      VARCHAR2(10) NOT NULL,  
    sessiondate    DATE NOT NULL,  
    sessiontime    DATE NOT NULL,  
    sessionclassid VARCHAR2(10) NOT NULL,  
    sessionstudioid VARCHAR2(10) NOT NULL,  
    sessioninstructorid VARCHAR2(10) NOT NULL  
);
```

```
ALTER TABLE "SESSIONS" ADD CONSTRAINT sessions_pk PRIMARY KEY ( sessionid );
```

```
CREATE TABLE studio (  
    studioid      VARCHAR2(10) NOT NULL,  
    studiocity    VARCHAR2(50),  
    address       VARCHAR2(200),  
    maxcap        NUMBER(4) NOT NULL,  
    studiophoneno NUMBER(10) NOT NULL,  
    studiocountry VARCHAR2(50)  
);
```

```
ALTER TABLE studio ADD CONSTRAINT studio_pk PRIMARY KEY ( studioid );
```

```
ALTER TABLE booking  
    ADD CONSTRAINT booking_customer_fk FOREIGN KEY ( bookingcustomerid )  
        REFERENCES customer ( customerid );
```

```
ALTER TABLE booking  
    ADD CONSTRAINT booking_session_fk FOREIGN KEY ( bookingsessionid )  
        REFERENCES "SESSION" ( sessionid );
```

```
ALTER TABLE customer  
    ADD CONSTRAINT customer_membership_fk FOREIGN KEY ( customermembershipid )  
        REFERENCES membership ( membershipid );
```

```
ALTER TABLE instructor  
    ADD CONSTRAINT instructor_studio_fk FOREIGN KEY ( instructorstudioid )  
        REFERENCES studio ( studioid );
```

```
ALTER TABLE payment  
    ADD CONSTRAINT payment_customer_fk FOREIGN KEY ( paymentcustomerid )  
        REFERENCES customer ( customerid );
```

```
ALTER TABLE "SESSIONS"
```

```
ADD CONSTRAINT sessionclassfk1 FOREIGN KEY ( sessionclassid )  
REFERENCES class ( classid );
```

```
ALTER TABLE "SESSIONS"
```

```
ADD CONSTRAINT session_instructor_fk FOREIGN KEY ( sessioninstructorid )  
REFERENCES instructor ( instructorid );
```

```
ALTER TABLE "SESSIONS"
```

```
ADD CONSTRAINT session_studio_fk FOREIGN KEY ( sessionstudioid )  
REFERENCES studio ( studioid );
```

Appendix 4: INSERT STATEMENTS + SOLUTION TO THE PROBLEM ENCOUNTERED

```
INSERT INTO membership VALUES ('M1','Basic Plan','Access to 3 classes/week',49.99);  
INSERT INTO membership VALUES ('M2','Premium Plan','Unlimited classes',99.99);  
INSERT INTO membership VALUES ('M3','Elite Plan','Unlimited + personal training',149.99);  
INSERT INTO membership VALUES ('M4','Student Plan','Discounted student access',39.99);  
INSERT INTO membership VALUES ('M5','Weekend Plan','Weekend-only access',29.99);  
INSERT INTO membership VALUES ('M6','Virtual Plan','Online classes only',59.99);  
INSERT INTO membership VALUES ('M7','Family Plan','Up to 4 members',199.99);  
INSERT INTO membership VALUES ('M8','Trial Plan','1-week trial',9.99);  
INSERT INTO membership VALUES ('M9','Corporate Plan','Company-sponsored',79.99);  
INSERT INTO membership VALUES ('M10','VIP Plan','All access + priority booking',249.99);
```

```
INSERT INTO customer VALUES ('C1','Emma Stone',5135551111,'M2');  
INSERT INTO customer VALUES ('C2','Olivia Brown',5135552222,'M1');  
INSERT INTO customer VALUES ('C3','Sophia Lee',5135553333,'M3');  
INSERT INTO customer VALUES ('C4','Ava Patel',5135554444,'M4');  
INSERT INTO customer VALUES ('C5','Isabella Kim',5135555555,'M5');  
INSERT INTO customer VALUES ('C6','Mia Davis',5135556666,'M6');  
INSERT INTO customer VALUES ('C7','Charlotte Wilson',5135557777,'M7');  
INSERT INTO customer VALUES ('C8','Amelia White',5135558888,'M8');  
INSERT INTO customer VALUES ('C9','Harper Hall',5135559999,'M9');  
INSERT INTO customer VALUES ('C10','Evelyn Scott',5135550000,'M10');
```

```
INSERT INTO studio VALUES ('S1','New York','123 Manhattan Ave',30,2125551111,'USA');  
INSERT INTO studio VALUES ('S2','Los Angeles','456 Sunset Blvd',25,3105552222,'USA');  
INSERT INTO studio VALUES ('S3','Miami','789 Ocean Dr',20,3055553333,'USA');  
INSERT INTO studio VALUES ('S4','Chicago','321 Lake Shore Dr',35,3125554444,'USA');
```

```

INSERT INTO studio VALUES ('S5','Dallas','654 Elm St',40,2145555555,'USA');
INSERT INTO studio VALUES ('S6','Boston','111 Beacon St',20,6175556666,'USA');
INSERT INTO studio VALUES ('S7','Seattle','222 Pine St',25,2065557777,'USA');
INSERT INTO studio VALUES ('S8','San Francisco','333 Market St',30,4155558888,'USA');
INSERT INTO studio VALUES ('S9','Denver','444 Colfax Ave',20,7205559999,'USA');
INSERT INTO studio VALUES ('S10','Atlanta','555 Peachtree St',35,4045550000,'USA');

```

```

INSERT INTO class VALUES ('CL1','Cardio Sculpt','High intensity cardio','Advanced',60,'Dumbbells');
INSERT INTO class VALUES ('CL2','Dance Cardio','Dance-based workout','Beginner',45,'None');
INSERT INTO class VALUES ('CL3','Pilates','Core strength','Intermediate',50,'Mat');
INSERT INTO class VALUES ('CL4','Yoga Flow','Flexibility and breathing','Beginner',60,'Mat');
INSERT INTO class VALUES ('CL5','HIIT','High intensity interval','Advanced',30,'None');
INSERT INTO class VALUES ('CL6','Barre','Low impact sculpt','Intermediate',55,'Barre');
INSERT INTO class VALUES ('CL7','Strength Training','Muscle building','Advanced',60,'Weights');
INSERT INTO class VALUES ('CL8','Stretch','Recovery session','Beginner',40,'');
INSERT INTO class VALUES ('CL9','Core Blast','Ab focused','Intermediate',45,'Mat');
INSERT INTO class VALUES ('CL10','Full Body Burn','Total workout','Advanced',60,'Dumbbells');

```

```

INSERT INTO instructor VALUES ('I1','Tracy Anderson','S1');
INSERT INTO instructor VALUES ('I2','Lily Adams','S2');
INSERT INTO instructor VALUES ('I3','Sophia Green','S3');
INSERT INTO instructor VALUES ('I4','Megan Fox','S4');
INSERT INTO instructor VALUES ('I5','Rachel White','S5');
INSERT INTO instructor VALUES ('I6','Emily Clark','S6');
INSERT INTO instructor VALUES ('I7','Olivia Turner','S7');
INSERT INTO instructor VALUES ('I8','Chloe Young','S8');
INSERT INTO instructor VALUES ('I9','Ava Scott','S9');
INSERT INTO instructor VALUES ('I10','Grace Hall','S10');

```

```

INSERT INTO "SESSIONS" VALUES ('SS1',DATE '2026-05-01',DATE '2026-05-01','CL1','S1','I1');
INSERT INTO "SESSIONS" VALUES ('SS2',DATE '2026-05-02',DATE '2026-05-02','CL2','S2','I2');
INSERT INTO "SESSIONS" VALUES ('SS3',DATE '2026-05-03',DATE '2026-05-03','CL3','S3','I3');
INSERT INTO "SESSIONS" VALUES ('SS4',DATE '2026-05-04',DATE '2026-05-04','CL4','S4','I4');
INSERT INTO "SESSIONS" VALUES ('SS5',DATE '2026-05-05',DATE '2026-05-05','CL5','S5','I5');

```

```

INSERT INTO "SESSIONS" VALUES ('SS6',DATE '2026-05-06',DATE
'2026-05-06','CL6','S6','I6');
INSERT INTO "SESSIONS" VALUES ('SS7',DATE '2026-05-07',DATE
'2026-05-07','CL7','S7','I7');
INSERT INTO "SESSIONS" VALUES ('SS8',DATE '2026-05-08',DATE
'2026-05-08','CL8','S8','I8');
INSERT INTO "SESSIONS" VALUES ('SS9',DATE '2026-05-09',DATE
'2026-05-09','CL9','S9','I9');
INSERT INTO "SESSIONS" VALUES ('SS10',DATE '2026-05-10',DATE
'2026-05-10','CL101','S10','I10');

```

```

INSERT INTO payment VALUES ('P1','Credit Card',DATE '2026-04-01',99.99,'C1');
INSERT INTO payment VALUES ('P2','Debit Card',DATE '2026-04-02',49.99,'C2');
INSERT INTO payment VALUES ('P3','PayPal',DATE '2026-04-03',149.99,'C3');
INSERT INTO payment VALUES ('P4','Credit Card',DATE '2026-04-04',39.99,'C4');
INSERT INTO payment VALUES ('P5','Cash',DATE '2026-04-05',29.99,'C5');
INSERT INTO payment VALUES ('P6','Credit Card',DATE '2026-04-06',59.99,'C6');
INSERT INTO payment VALUES ('P7','Debit Card',DATE '2026-04-07',199.99,'C7');
INSERT INTO payment VALUES ('P8','PayPal',DATE '2026-04-08',9.99,'C8');
INSERT INTO payment VALUES ('P9','Credit Card',DATE '2026-04-09',79.99,'C9');
INSERT INTO payment VALUES ('P10','Credit Card',DATE '2026-04-10',249.99,'C10');

```

```

INSERT INTO booking VALUES ('B101', DATE '2026-04-15', 'Confirmed', 'SS1', 'C2');
INSERT INTO booking VALUES ('B102', DATE '2026-04-16', 'Confirmed', 'SS2', 'C3');
INSERT INTO booking VALUES ('B103', DATE '2026-04-17', 'Pending', 'SS3', 'C4');
INSERT INTO booking VALUES ('B104', DATE '2026-04-18', 'Cancelled', 'SS4', 'C5');
INSERT INTO booking VALUES ('B105', DATE '2026-04-19', 'Confirmed', 'SS5', 'C6');
INSERT INTO booking VALUES ('B106', DATE '2026-04-20', 'Confirmed', 'SS6', 'C7');
INSERT INTO booking VALUES ('B107', DATE '2026-04-21', 'Pending', 'SS7', 'C8');
INSERT INTO booking VALUES ('B108', DATE '2026-04-22', 'Confirmed', 'SS8', 'C9');
INSERT INTO booking VALUES ('B109', DATE '2026-04-23', 'Cancelled', 'SS9', 'C10');
INSERT INTO booking VALUES ('B110', DATE '2026-04-24', 'Confirmed', 'SS10', 'C1');

```

```

SELECT sessionid FROM "SESSIONS";
ALTER TABLE booking DROP CONSTRAINT booking_session_fk;
SELECT * FROM "SESSIONS";
ALTER TABLE booking
ADD CONSTRAINT booking_session_fk
FOREIGN KEY (bookingsessionid)
REFERENCES "SESSIONS" (sessionid);
INSERT INTO booking
VALUES ('TEST1', DATE '2026-04-15', 'Confirmed', 'SS1', 'C2');
INSERT INTO booking

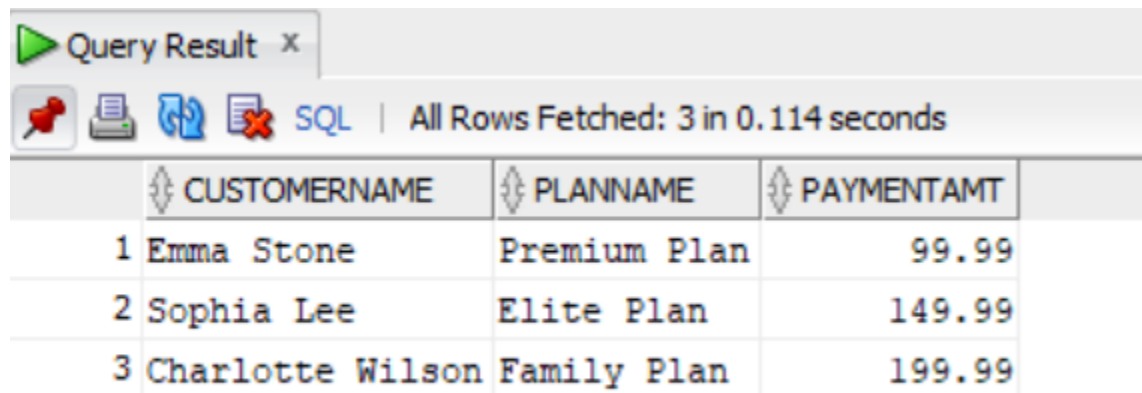
```

VALUES ('B101', DATE '2026-04-15', 'Confirmed', 'SS1', 'C2');

Appendix 5: SQL Queries

Query 1:

```
SELECT customername, planname, paymentamt
FROM customer JOIN membership ON customer.customermembershipid =
membership.membershipid
JOIN payment ON customer.customerid = payment.paymentcustomerid
WHERE paymentamt > (
    SELECT AVG(paymentamt)
    FROM payment
)
AND customer.customerid IN (
    SELECT bookingcustomerid
    FROM booking
    WHERE bookingstatus = 'Confirmed'
);
```



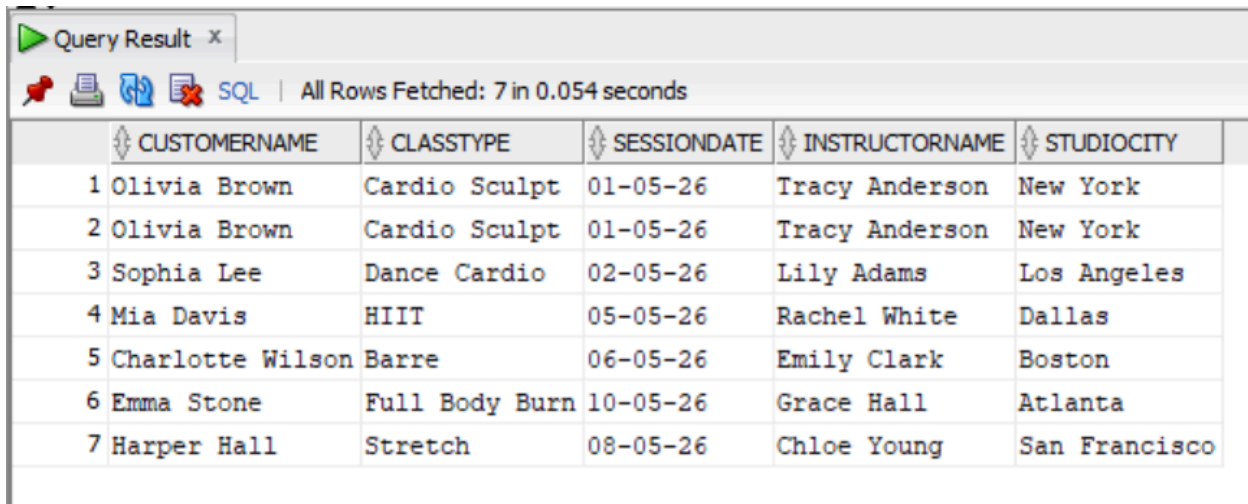
The screenshot shows a 'Query Result' window with a toolbar containing icons for a pin, print, copy, and delete, along with an 'SQL' label and a status bar indicating 'All Rows Fetched: 3 in 0.114 seconds'. The table below displays the results of the query.

	CUSTOMERNAME	PLANNAME	PAYMENTAMT
1	Emma Stone	Premium Plan	99.99
2	Sophia Lee	Elite Plan	149.99
3	Charlotte Wilson	Family Plan	199.99

Query 2:

```
SELECT customername, classtype, sessiondate, instructorname, studiocity
FROM booking JOIN customer ON booking.bookingcustomerid = customer.customerid
JOIN "SESSIONS" ON booking.bookingsessionid = sessions.sessionid
JOIN class ON sessions.sessionclassid = class.classid
JOIN instructor ON sessions.sessioninstructorid = instructor.instructorid
JOIN studio ON sessions.sessionstudioid = studio.studioid
```

WHERE bookingstatus = 'Confirmed';



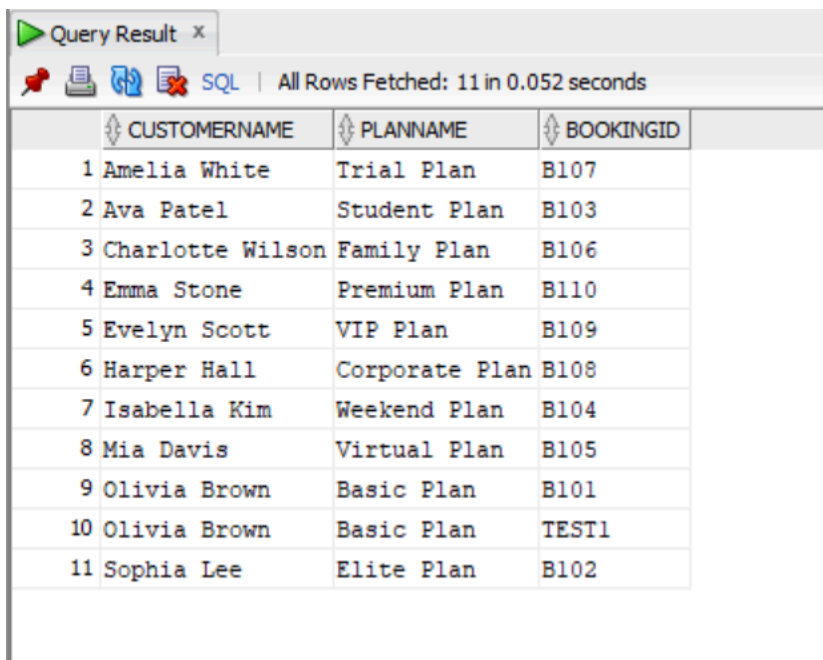
Query Result x

SQL | All Rows Fetched: 7 in 0.054 seconds

	CUSTOMERNAME	CLASSTYPE	SESSIONDATE	INSTRUCTORNAME	STUDIOCITY
1	Olivia Brown	Cardio Sculpt	01-05-26	Tracy Anderson	New York
2	Olivia Brown	Cardio Sculpt	01-05-26	Tracy Anderson	New York
3	Sophia Lee	Dance Cardio	02-05-26	Lily Adams	Los Angeles
4	Mia Davis	HIIT	05-05-26	Rachel White	Dallas
5	Charlotte Wilson	Barre	06-05-26	Emily Clark	Boston
6	Emma Stone	Full Body Burn	10-05-26	Grace Hall	Atlanta
7	Harper Hall	Stretch	08-05-26	Chloe Young	San Francisco

Query 3:

```
SELECT customername, planname, booking.bookingid
FROM customer JOIN membership ON customer.customermembershipid =
membership.membershipid
LEFT OUTER JOIN booking ON customer.customerid = booking.bookingcustomerid
ORDER BY customername;
```



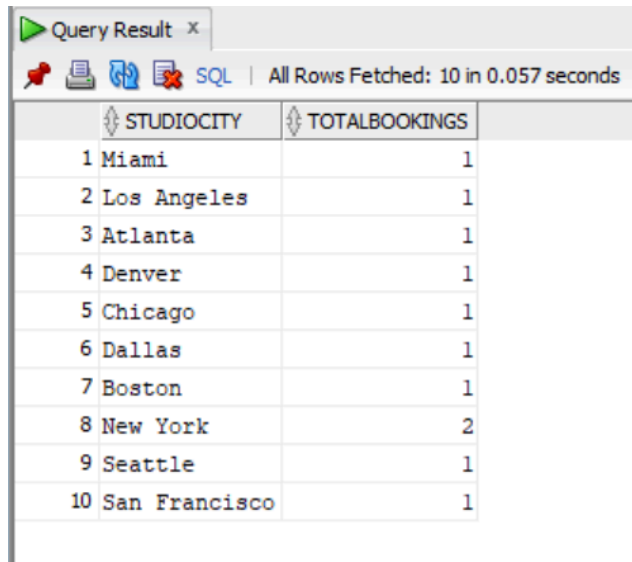
Query Result x

SQL | All Rows Fetched: 11 in 0.052 seconds

	CUSTOMERNAME	PLANNAME	BOOKINGID
1	Amelia White	Trial Plan	B107
2	Ava Patel	Student Plan	B103
3	Charlotte Wilson	Family Plan	B106
4	Emma Stone	Premium Plan	B110
5	Evelyn Scott	VIP Plan	B109
6	Harper Hall	Corporate Plan	B108
7	Isabella Kim	Weekend Plan	B104
8	Mia Davis	Virtual Plan	B105
9	Olivia Brown	Basic Plan	B101
10	Olivia Brown	Basic Plan	TEST1
11	Sophia Lee	Elite Plan	B102

Query 4:

```
SELECT studiocity, COUNT(booking.bookingid) AS totalbookings
FROM studio JOIN "SESSIONS" ON studio.studioid = sessions.sessionstudioid
JOIN booking ON sessions.sessionid = booking.bookingsessionId
GROUP BY studiocity;
```

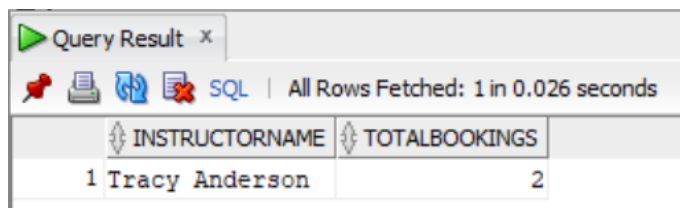


The screenshot shows a SQL query result window titled "Query Result". It displays the results of a query that counts bookings by studio city. The window shows 10 rows of data, with columns "STUDIOCITY" and "TOTALBOOKINGS". The data is as follows:

STUDIOCITY	TOTALBOOKINGS
1 Miami	1
2 Los Angeles	1
3 Atlanta	1
4 Denver	1
5 Chicago	1
6 Dallas	1
7 Boston	1
8 New York	2
9 Seattle	1
10 San Francisco	1

--Query 5

```
SELECT instructorname, COUNT(booking.bookingid) AS totalbookings
FROM instructor JOIN "SESSIONS" ON instructor.instructorid = sessions.sessioninstructorid
JOIN booking ON sessions.sessionid = booking.bookingsessionId
WHERE bookingstatus = 'Confirmed'
GROUP BY instructorname
HAVING COUNT(booking.bookingid) > 1;
```



The screenshot shows a SQL query result window titled "Query Result". It displays the results of a query that counts confirmed bookings by instructor name. The window shows 1 row of data, with columns "INSTRUCTORNAME" and "TOTALBOOKINGS". The data is as follows:

INSTRUCTORNAME	TOTALBOOKINGS
1 Tracy Anderson	2

--Query 6

```
CREATE INDEX indexbookingcustomer ON booking(bookingcustomerid);
CREATE INDEX indexbookingsession ON booking(bookingsessionid);
CREATE INDEX indexpaymentcustomer ON payment(paymentcustomerid);
```

--Query 7

```
CREATE VIEW customeractivityview AS
```

```
SELECT customername, planname, bookingstatus, paymentamt
FROM customer JOIN membership ON customer.customermembershipid =
membership.membershipid
LEFT JOIN booking ON customer.customerid = booking.bookingcustomerid
JOIN payment ON customer.customerid = payment.paymentcustomerid;
```

--Query 8

```
CREATE OR REPLACE PROCEDURE cancelbookingandrefund (
    refundbookingid VARCHAR2,
    refundcustomerid VARCHAR2
) AS
BEGIN
```

```
    UPDATE booking
    SET bookingstatus = 'Cancelled'
    WHERE bookingid = refundbookingid;
    UPDATE payment
    SET paymentamt = paymentamt - 10
    WHERE paymentcustomerid = refundcustomerid;
END;
/
```

--Query 9

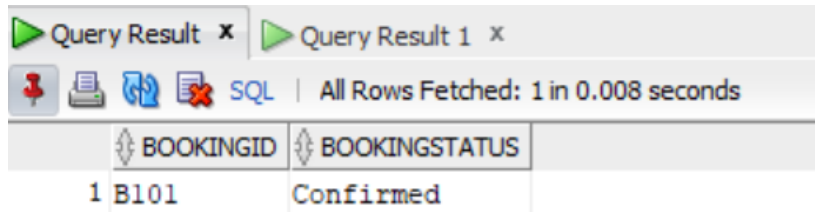
```
CREATE OR REPLACE TRIGGER validateinstructorstudio
BEFORE INSERT ON "SESSIONS"
FOR EACH ROW
DECLARE
    vinstructorstudio VARCHAR2(10);
BEGIN
```

```
    SELECT instructorstudioid
    INTO vinstructorstudio
    FROM instructor
    WHERE instructorid = :NEW.sessioninstructorid;
```

```
    IF vinstructorstudio <> :NEW.sessionstudioid THEN
        RAISE_APPLICATION_ERROR(-20004, 'Instructor does not belong to this studio');
    END IF;
END;
/
```

Appendix 6: Checks for Procedure, Trigger, & View

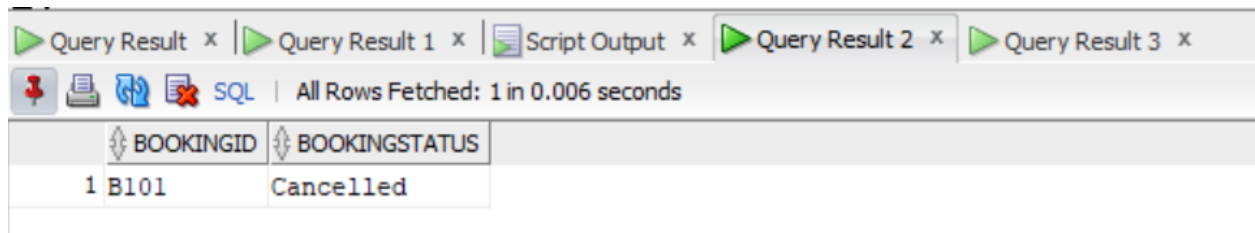
Before Procedure



The screenshot shows a SQL query result window with two tabs: 'Query Result' and 'Query Result 1'. The 'Query Result' tab is active, displaying a table with two columns: 'BOOKINGID' and 'BOOKINGSTATUS'. The table contains one row with the values '1 B101' and 'Confirmed'. The status bar indicates 'All Rows Fetched: 1 in 0.008 seconds'.

BOOKINGID	BOOKINGSTATUS
1 B101	Confirmed

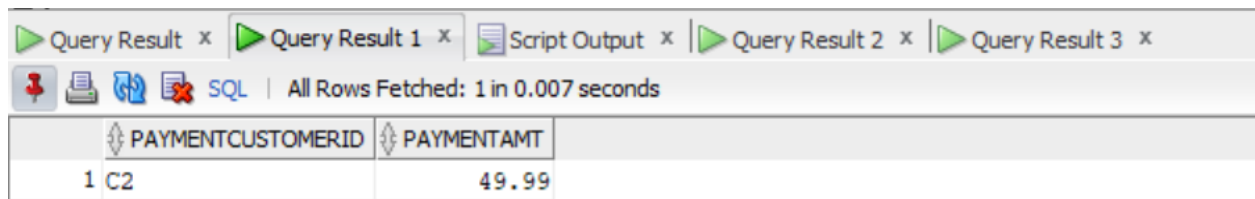
After the procedure was executed



The screenshot shows a SQL query result window with five tabs: 'Query Result', 'Query Result 1', 'Script Output', 'Query Result 2', and 'Query Result 3'. The 'Query Result' tab is active, displaying a table with two columns: 'BOOKINGID' and 'BOOKINGSTATUS'. The table contains one row with the values '1 B101' and 'Cancelled'. The status bar indicates 'All Rows Fetched: 1 in 0.006 seconds'.

BOOKINGID	BOOKINGSTATUS
1 B101	Cancelled

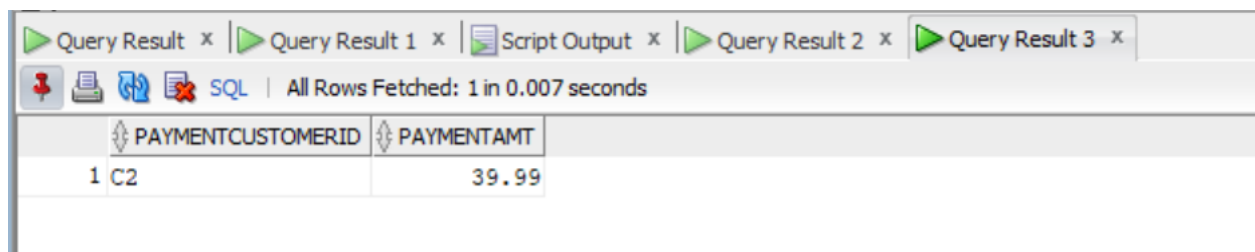
Before procedure



The screenshot shows a SQL query result window with five tabs: 'Query Result', 'Query Result 1', 'Script Output', 'Query Result 2', and 'Query Result 3'. The 'Query Result' tab is active, displaying a table with two columns: 'PAYMENTCUSTOMERID' and 'PAYMENTAMT'. The table contains one row with the values '1 C2' and '49.99'. The status bar indicates 'All Rows Fetched: 1 in 0.007 seconds'.

PAYMENTCUSTOMERID	PAYMENTAMT
1 C2	49.99

After Procedure



The screenshot shows a SQL query result window with five tabs: 'Query Result', 'Query Result 1', 'Script Output', 'Query Result 2', and 'Query Result 3'. The 'Query Result' tab is active, displaying a table with two columns: 'PAYMENTCUSTOMERID' and 'PAYMENTAMT'. The table contains one row with the values '1 C2' and '39.99'. The status bar indicates 'All Rows Fetched: 1 in 0.007 seconds'.

PAYMENTCUSTOMERID	PAYMENTAMT
1 C2	39.99

Query Used:

```
SELECT bookingid, bookingstatus  
FROM booking  
WHERE bookingid = 'B101';
```

```
SELECT paymentcustomerid, paymentamt  
FROM payment
```

```

WHERE paymentcustomerid = 'C2';

BEGIN
    cancelbookingandrefund('B101', 'C2');
END;
/

```

```

SELECT bookingid, bookingstatus
FROM booking
WHERE bookingid = 'B101';

```

```

SELECT paymentcustomerid, paymentamt
FROM payment
WHERE paymentcustomerid = 'C2';

```

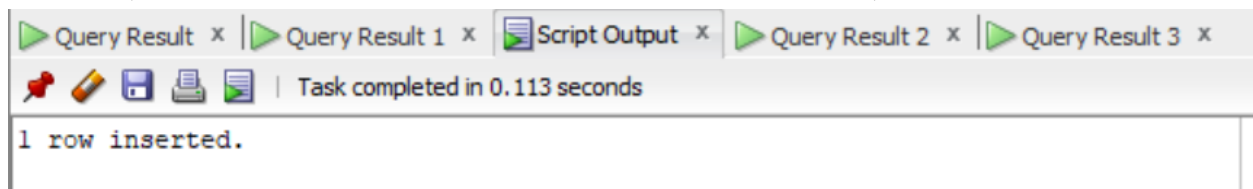
Trigger Check with Valid and Invalid Case

--Valid Case

```

INSERT INTO "SESSIONS"
VALUES ('SS11', DATE '2026-05-11', DATE '2026-05-11', 'CL1', 'S1', 'I1');

```



--Invalid Case

```

INSERT INTO "SESSIONS"
VALUES ('SS12', DATE '2026-05-12', DATE '2026-05-12', 'CL1', 'S2', 'I1');

```

```

Error starting at line : 356 in command -
INSERT INTO "SESSIONS"
VALUES ('SS12', DATE '2026-05-12', DATE '2026-05-12', 'CL1', 'S2', 'I1')
Error at Command Line : 356 Column : 13
Error report -
SQL Error: ORA-20004: Instructor does not belong to this studio
ORA-06512: at "SHAHKH2.VALIDATEINSTRUCTORSTUDIO", line 12
ORA-04088: error during execution of trigger 'SHAHKH2.VALIDATEINSTRUCTORSTUDIO'

```

VIEW CHECK

Query Result x	Query Result 1 x	Script Output x	Query Result 2 x	Query Result 3 x	Query Result 4 x
SQL All Rows Fetched: 11 in 0.015 seconds					
	CUSTOMERNAME	PLANNAME	BOOKINGSTATUS	PAYMENTAMT	
1	Olivia Brown	Basic Plan	Confirmed	39.99	
2	Olivia Brown	Basic Plan	Cancelled	39.99	
3	Sophia Lee	Elite Plan	Confirmed	149.99	
4	Ava Patel	Student Plan	Pending	39.99	
5	Isabella Kim	Weekend Plan	Cancelled	29.99	
6	Mia Davis	Virtual Plan	Confirmed	59.99	
7	Charlotte Wilson	Family Plan	Confirmed	199.99	
8	Amelia White	Trial Plan	Pending	9.99	
9	Harper Hall	Corporate Plan	Confirmed	79.99	
10	Evelyn Scott	VIP Plan	Cancelled	249.99	
11	Emma Stone	Premium Plan	Confirmed	99.99	

```
SELECT *
FROM customeractivityview;
```